AI Code Review Platform: Full Project Spec

Nov 10, 2026 · @mo

1. Project overview

The AI Code Review Platform is a GitHub App that reviews every pull request automatically, using the whole codebase as context, and posts its findings as inline comments and a GitHub check run.

Problem. Human review is slow and inconsistent, and small teams often skip it. Most AI reviewers only see the diff, so their comments are shallow and noisy.

Solution. When a PR opens or updates, the system verifies the webhook, queues a job, fetches the diff, retrieves related code from a vector index of the repo, runs specialist agents (bug, security, test, quality) that can call tools, then merges, dedupes and ranks the findings before posting them.

Simple ga: PR vasthe, AI motham codebase telusukoni review chesi, comments petthundi.

Goals

1. Work end to end on a real GitHub repo, deployed with a live demo.
2. Give context-aware reviews that use retrieved code, not only the diff.
3. Keep noise low with confidence thresholds, dedupe and a feedback loop.
4. Be measurable: an evaluation suite reporting precision, recall and false-positive rate.
5. Show production-style backend work: idempotent webhooks, queues, retries, multi-tenancy, observability, CI/CD.

Non-goals for version 1

- No auto-merge and no auto-fix commits. The bot comments only.
- No IDE plugin.
- Only JavaScript/TypeScript and Python are supported at first.
- No model training. It uses hosted LLM and embedding APIs.

What makes it stand out

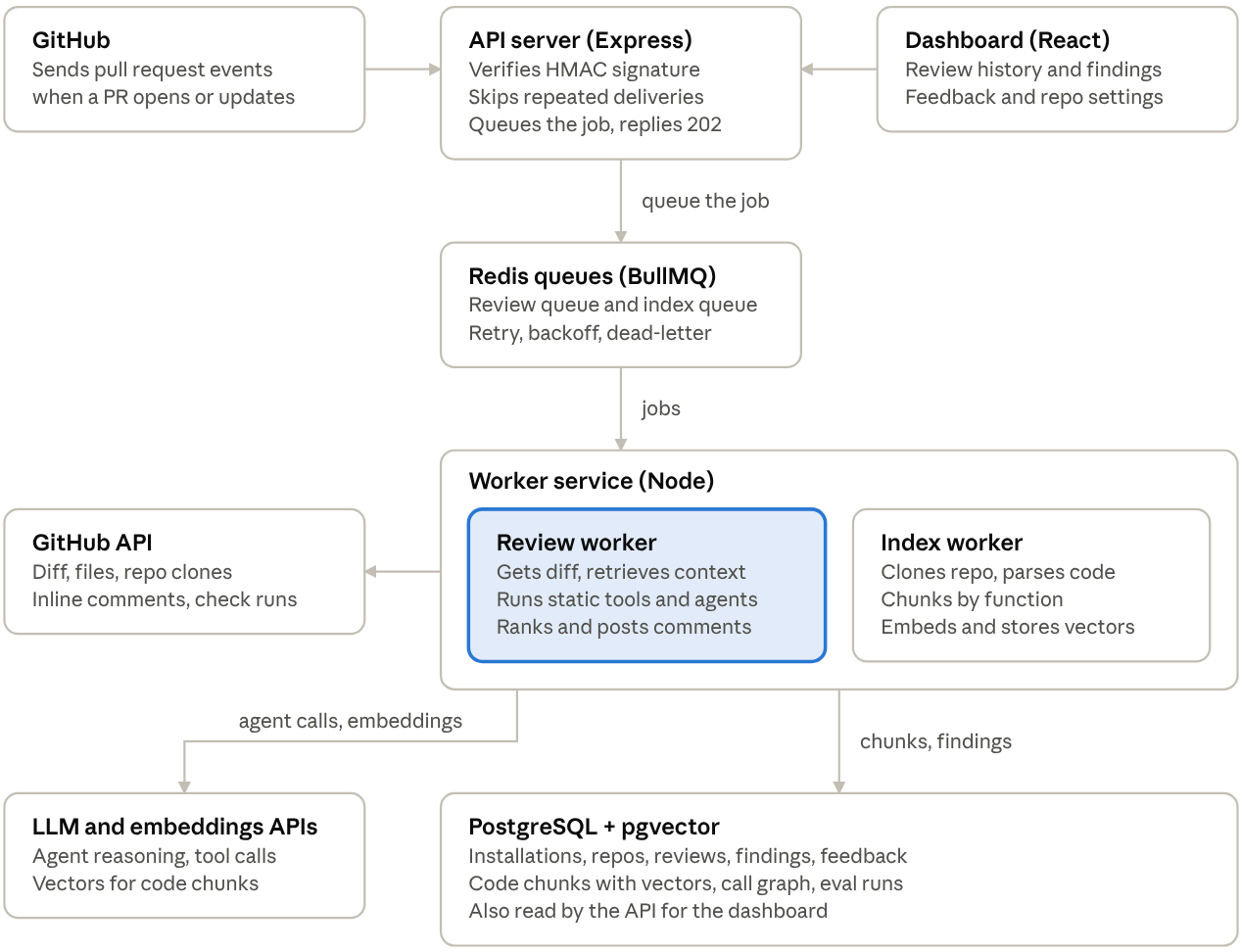
- **Measured quality.** Eval numbers on a benchmark of PRs with known bugs.

- **Backend depth.** Idempotency, retries, dead-letter queue, stale-review cancel, rate-limit handling.
- **Hybrid analysis.** Static tools (ESLint, Semgrep, Bandit) run first and the LLM explains and prioritizes their output.
- **Feedback loop.** Thumbs up or down on comments tunes what gets posted.
- **Context.** Reviews use callers, callees and similar code, not only the changed lines.

2. System architecture

A webhook is verified and queued in under a second, and a separate worker does the slow review work, so GitHub never waits on the LLM.

A webhook becomes a queued job that a worker reviews



system architecture · 8 components, 7 connections

Read it top to bottom: GitHub's event enters through the API server, waits as a job in Redis, and is picked up by a worker. The accented Review worker is where the review happens; it calls GitHub, the LLM and Postgres.

Component	Responsibility	Why it is separate
API server	Verify the signature, dedupe, enqueue, serve the dashboard API	It must answer GitHub within seconds, so it does no slow work
Redis queues	Hold review and index jobs, with retries and a dead-letter queue	Jobs survive a crash and can be retried
Review worker	Diff, retrieval, static tools, agents, posting	LLM calls are slow and must not block the API
Index worker	Clone, parse, chunk, embed, store	Runs on install and push, not on every PR, with a different load
PostgreSQL + pgvector	Relational data and the vector index	One store to run, back up and secure
LLM and embeddings APIs	Agent reasoning, tool calls and vectors	External and rate-limited, so wrap them behind your own interface
Dashboard	Review history, findings, feedback and settings	A view with feedback buttons; it holds no review logic

3. Tech stack

Use TypeScript on Node.js for the whole backend, with Zod validating all outside data, and pick one choice per layer from the table below.

Layer	Choice	Why
Language	TypeScript (`strict: true`) on Node.js 20+	GitHub payloads and LLM findings are deeply nested; types catch typos and share one `Finding` type across API, worker and dashboard. Plain JS + Zod + JSDoc is the fallback.

Layer	Choice	Why
API server	Express (Fastify is fine too)	You already know Express. Needs `express.raw()` on the webhook route so the HMAC is computed on the exact bytes.
GitHub	Octokit (`@octokit/app` , `@octokit/rest` , `@octokit/webhooks`)	Official, typed, handles App auth and token caching.
Queue	BullMQ + Redis	Retries, backoff, delays, concurrency, job IDs for dedupe, built in.
Database	PostgreSQL 16 + pgvector	One database for relational data and vector search. Fewer moving parts to explain and run.
ORM / SQL	Drizzle ORM	Typed queries, migrations, and pgvector support. Plain `pg` + SQL files also works.
Validation	Zod	Runtime checks on webhooks, queue jobs, env vars and LLM output; the TypeScript type comes from the same schema.
Code parsing	Tree-sitter (`tree-sitter` , `tree-sitter-typescript` , `tree-sitter-python`)	Real syntax trees, so chunks are whole functions and classes.
Embeddings	One hosted embeddings API (OpenAI, Voyage, or similar)	Pick one model and record its dimension; changing it means re-indexing everything.
LLM	Any API with tool calling (Claude, OpenAI, Gemini) behind your own `LlmClient` interface	Lets you swap models and compare them in the eval suite.
Agents	LangGraph (JS)	Graph of agents with shared state; you already know the Python version.

Layer	Choice	Why
Static analysis	ESLint, Semgrep, Bandit, Ruff	Cheap, precise signals that the LLM then explains and prioritizes.
Dashboard	React + Vite	Matches your existing React skills.
Logging / metrics	pino, prom-client, Sentry (optional OpenTelemetry)	Structured logs with a review ID, metrics for queue depth and latency.
Testing	Vitest, Supertest, Testcontainers, nock or msw	Unit, API and integration tests with real Postgres and Redis.
Infra	Docker, docker-compose, GitHub Actions, GHCR	Same containers locally, in CI and in production.
Hosting	Render, Railway or Fly.io; or one AWS EC2 box with docker-compose	See the DevOps section for the trade-offs.

Decision rule. One language for the whole backend. If TypeScript slows you down for more than two days, switch to plain JS with Zod and keep the architecture unchanged.

4. Repo and folder structure

Use one pnpm monorepo where each folder maps to one box in the architecture, so the code is as easy to explain as the diagram.

```
ai-code-reviewer/
├── apps/
│   ├── api/                 # Express: webhooks, REST for dashboard, GitHub
OAuth
│   │   └── src/{server.ts, routes/, middleware/verifySignature.ts}
│   ├── worker/              # BullMQ workers: review worker, index worker
│   │   └── src/{review.worker.ts, index.worker.ts, pipeline/}
│   └── dashboard/           # React + Vite
├── packages/
│   ├── core/                # shared types, Zod schemas, config loader,
logger, errors
│   ├── github/              # App auth, token cache, diff, review comments,
check runs
│   ├── db/                  # Drizzle schema, migrations, query helpers
│   ├── indexer/             # clone, parse (Tree-sitter), chunk, embed,
```

```
upsert
|   ├── retrieval/           # vector + keyword search, call graph, context
builder
|   ├── agents/              # LangGraph graph, prompts, tools
|   ├── analyzers/           # ESLint / Semgrep / Bandit runners + output
normalizers
|   └── llm/                 # LlmClient interface + provider adapters
├── evals/                   # dataset/, runner.ts, metrics.ts, reports/
├── infra/                   # Dockerfiles, docker-compose.yml, deploy
config
├── docs/                    # architecture.md, adr/, screenshots
└── .github/workflows/       # ci.yml, eval.yml, deploy.yml
```

Rules that keep it clean

1. Apps depend on packages. Packages never import from apps.
2. Only `packages/github` talks to GitHub, only `packages/db` talks to Postgres, only `packages/llm` talks to an LLM provider.
3. Every package has its own `tests/` folder and a small public `index.ts`.
4. Shared types live in `packages/core`. A type used in two places is defined once.
5. Write a short ADR (architecture decision record) in `docs/adr/` for each big choice, such as why pgvector over Pinecone. Interviewers like these.

5. Database design

PostgreSQL with the pgvector extension holds everything: tenants, reviews, findings, feedback and the code index, so one backup and one connection pool cover the whole system.

Table	Key columns	Purpose
`installations`	`id` (GitHub installation id), `account_login`, `account_type`, `suspended_at`	One row per GitHub account that installed the app. This is the tenant boundary.
`repos`	`id`, `installation_id`, `github_repo_id`, `full_name`, `default_branch`, `last_indexed_sha`, `index_status`, `settings` (jsonb)	Repos the app can see, their index state and per-repo settings.

Table	Key columns	Purpose
`webhook_deliveries`	`delivery_id` (primary key), `event`, `received_at`, `processed_at`, `status`	Idempotency: a repeated `X-GitHub-Delivery` id is rejected by the primary key.
`reviews`	`id`, `repo_id`, `pr_number`, `head_sha`, `status`, `started_at`, `finished_at`, `model`, `prompt_version`, `tokens_in`, `tokens_out`, `cost_usd`, `error`	One row per review run. Unique on (`repo_id`, `pr_number`, `head_sha`). Status: queued, running, done, failed, cancelled.
`findings`	`id`, `review_id`, `agent`, `file_path`, `line_start`, `line_end`, `severity`, `category`, `title`, `message`, `suggestion`, `confidence`, `fingerprint`, `status`, `github_comment_id`	Every finding, including suppressed and duplicate ones, so you can analyze noise later.
`feedback`	`id`, `finding_id`, `user_login`, `value` (+1 or -1), `reason`, `created_at`	Helpful or not-helpful signals that tune future reviews.
`code_chunks`	`id`, `repo_id`, `file_path`, `symbol_name`, `symbol_kind`, `language`, `start_line`, `end_line`, `content`, `content_hash`, `commit_sha`, `embedding vector(N)`, `tsv tsvector`	The searchable code index. `N` = your embedding model's dimension.
`file_index`	`repo_id`, `file_path`, `file_hash`, `indexed_sha`	Lets incremental indexing skip unchanged files.
`symbol_edges`	`repo_id`, `from_chunk_id`, `to_symbol_name`, `kind` (calls or imports)	A light call graph for finding callers and callees.
`eval_runs`	`id`, `git_sha`, `dataset_version`, `model`, `prompt_version`, `precision`, `recall`, `fp_rate`, `avg_cost_usd`, `created_at`	History of evaluation results, shown on the dashboard.

Table	Key columns	Purpose
`users`	`id`, `github_user_id`, `login`, `created_at`	Dashboard users, signed in through GitHub OAuth.

Indexes and constraints

- HNSW index on `code_chunks.embedding` with cosine distance, plus a GIN index on `tsv` for keyword search.
- B-tree on (`repo_id`, `file_path`) for `code_chunks` and `findings`.
- Unique (`repo_id`, `pr_number`, `head_sha`) on `reviews` and unique `fingerprint` per review on `findings`.
- Foreign keys with `ON DELETE CASCADE` from `installations` down, so uninstalling the app deletes that tenant's data in one statement.

Tenant isolation. Every query filters by `repo_id` or `installation_id`. For a hard guarantee, turn on Postgres row-level security and set the current installation per connection, so a missing `WHERE` returns nothing instead of leaking another tenant's code.

6. GitHub App setup and authentication

Build this as a GitHub App, not an OAuth app or a personal token, because an App gets per-repo permissions, webhooks and short-lived tokens.

Register the app

In GitHub go to Settings, Developer settings, GitHub Apps, New GitHub App. Create two apps: one for development (pointing at a tunnel URL) and one for production.

Setting	Value
Webhook URL	`https://<your-host>/webhooks/github` (in dev, a smee.io or cloudflared tunnel URL)
Webhook secret	A long random string, stored as `GITHUB_WEBHOOK_SECRET`
Repository permissions	Contents: read. Pull requests: read and write. Checks: read and write. Metadata: read.

Setting	Value
Subscribe to events	`pull_request`, `push`, `installation`, `installation_repositories`, `pull_request_review_thread`
Private key	Generate and download the `.pem`; store it as a base64 env var
Where it can be installed	Any account (production), only your account (development)

Ask for the minimum permissions. You can add more later, and users must approve each addition.

Token flow

1. **App JWT.** Sign a JWT with the private key using RS256. Set `iss` to the App ID, `iat` to now minus 60 seconds, and `exp` to at most 10 minutes ahead.
2. **Installation token.** Call `POST /app/installations/{installation_id}/access_tokens` with that JWT. The response is a token valid for about 1 hour. You can narrow it to specific repos and permissions.
3. **Use the token** for the REST API calls and for `git clone https://x-access-token:<token>@github.com/<owner>/<repo>.git`.
4. **Cache it** per installation id (memory or Redis with a TTL) and refresh when under 5 minutes remain. `@octokit/auth-app` does this for you; read how it works so you can explain it.

Simple ga: App JWT ante 'nenu ee app ni' ani chupinche ID. Installation token ante 'ee repo lo ee pani cheyochu' ani GitHub ichche 1 gantha pass.

Install lifecycle

- `installation.created` and `installation_repositories.added`: create rows in `installations` and `repos`, then queue a full index job.
- `installation.deleted` and `installation_repositories.removed`: delete that tenant's repos, chunks, reviews and findings.
- `installation.suspend`: mark the installation suspended and skip its jobs.

Dashboard login

Use the GitHub App's user-to-server OAuth flow. After login, call the installations API for that user to learn which repos they may see, and issue an httpOnly, secure, SameSite session cookie. The API must check repo access on every request.

Secrets

Keep `GITHUB_APP_ID`, `GITHUB_PRIVATE_KEY_BASE64`, `GITHUB_WEBHOOK_SECRET`, OAuth client credentials and LLM keys in environment variables or a secret manager. Validate them with Zod at startup so the app fails fast if one is missing. Never commit them and never log them.

7. Webhooks and queue

The webhook endpoint does only four things (verify, dedupe, filter, enqueue) and returns in under a second, because GitHub gives up on a slow webhook after about 10 seconds and LLM work takes minutes.

Webhook handler, in order

1. **Raw body.** Mount `express.raw({ type: 'application/json' })` on this route. If the body is parsed first, the HMAC no longer matches.
2. **Verify the signature.** Compute HMAC-SHA256 of the raw body with the webhook secret, compare to `X-Hub-Signature-256` with `crypto.timingSafeEqual`, and return 401 on mismatch.
3. **Read headers.** `X-GitHub-Event` (event type) and `X-GitHub-Delivery` (unique id).
4. **Idempotency.** Insert the delivery id into `webhook_deliveries` with `ON CONFLICT DO NOTHING`. If nothing was inserted, this is a repeat: return 200 and stop. GitHub can and does redeliver.
5. **Validate with Zod.** Parse only the fields you need from the payload.
6. **Filter.** Handle `pull_request` actions `opened`, `synchronize`, `reopened` and `ready_for_review`. Ignore bot authors, optionally ignore drafts, and skip PRs over your size limit.
7. **Enqueue and return 202.** Add a BullMQ job with `jobId = repoId:prNumber:headSha` so the same commit is never queued twice.

Simple ga: Webhook ni doorbell anukondi. Gate daggara vachina vallani identity check chesi (signature), already vachina vallani malli lopaliki rakunda (idempotency), pani ni queue lo petti, ventane 'sare' ani cheppali.

Queue design

Item	Design
Queues	`review` and `index`, each with its own worker and concurrency
Job payload	`installationId`, `repoId`, `repoFullName`, `prNumber`, `headSha`, `baseSha`, `deliveryId`
Debounce	Delay review jobs by 10 to 20 seconds so rapid pushes collapse into one review
Concurrency	A few jobs at a time per worker, tuned to LLM and GitHub rate limits
Retries	5 attempts with exponential backoff and jitter
Retryable errors	HTTP 429, 5xx, network timeouts, LLM timeouts
Non-retryable errors	Revoked installation (401), deleted repo (404), invalid payload. Throw BullMQ's `UnrecoverableError` so they fail at once.
Dead-letter	After the last attempt, copy the job to a `review-dlq` queue, set the review to `failed`, and post a neutral check run saying the review failed
Shutdown	On SIGTERM call `worker.close()` so the running job finishes before the process exits

Cancelling stale reviews

When a new commit is pushed while a review is running, the old review is wasted work and may post comments on code that no longer exists.

1. On a new `synchronize` event, find any `queued` or `running` review for the same PR with an older `head_sha`.
2. Mark it `cancelled` and set a Redis key `cancel:<reviewId>`.

3. The worker checks that key between pipeline stages and passes an `AbortSignal` to LLM and HTTP calls, so it stops quickly.
4. Before posting anything, re-fetch the PR and confirm `head_sha` still matches. If not, discard the results.

This is a strong interview point because most simple bots skip it.

8. Worker pipeline

The review worker runs thirteen stages in order, checks for cancellation between stages, and keeps partial results so one failing stage does not lose the whole review.

1. **Start.** Load the job, set the `reviews` row to `running`, and check the cancel flag.
2. **Token.** Get the cached installation token for this repo.
3. **Check run.** Create a GitHub check run named 'AI Code Review' with status `in_progress`, so the PR shows that a review is under way.
4. **Fetch the change.** Call `GET /pulls/{n}/files` (paginated) for the changed files and fetch each file's content at `head_sha`. Large files may have no `patch` field, so handle that case.
5. **Filter files.** Skip lockfiles, generated code, vendored folders, minified or binary files and anything over the size limit. Apply the repo's ignore globs from `settings`.
6. **Map diff lines.** Parse each patch into hunks and record which right-side line numbers are in the diff. GitHub only accepts inline comments on lines inside the diff.
7. **Static analysis.** Check out the head commit in a temporary directory and run ESLint, Semgrep, Bandit or Ruff on the changed files with your own configs. Normalize each result into a candidate finding.
8. **Retrieve context.** For each changed symbol, build a context pack (see the next section).
9. **Run agents.** Run the LangGraph graph with the diff, context and static results (see the agents section).
10. **Post-process.** Validate with Zod, drop low-confidence findings, dedupe, rank, cap the count, and remove findings on lines outside the diff or already posted on this PR.
11. **Guard.** Confirm the review is not cancelled and the PR's `head_sha` has not changed.
12. **Publish.** Send one pull-request review request containing all inline comments plus a summary body. Update the check run with a conclusion of `neutral` and a markdown summary. Version 1 never blocks merging.

13. **Persist and clean up.** Save findings, token counts, cost and timings, emit metrics, delete the temporary directory, set the review to `done`.

Budgets. Set a time limit for the whole review (for example 5 minutes), a token limit per agent call, and a file limit per PR. When a PR is too large, review the highest-risk files and say so in the summary.

Failure behavior. If static analysis fails, continue with the LLM only. If one agent fails, post the other agents' findings and note the gap. If publishing fails, retry the job; findings are already saved, so the retry does not call the LLM again.

9. Indexing and RAG

The indexer turns a repo into function-level chunks with embeddings, and the retrieval layer uses that index to hand each agent the code it needs to judge a change.

Simple ga: Embeddings ante code ni 'meaning' numbers ga marchadam. Vector DB lo aa numbers pettukuntaru. PR vachinappudu, changed code ki similar and related code ni vethiki AI ki istharu. Open-book exam lo correct pages matrame chupinchinattu.

Indexing pipeline

1. **Trigger.** A full index on install, an incremental index on each push to the default branch.
2. **Clone.** Shallow clone (`--depth 1`) into a temporary directory with the installation token. List files with `git ls-files`.
3. **Filter.** Keep supported extensions, skip vendored, generated and oversized files (for example over 200 KB).
4. **Parse.** Use Tree-sitter to extract functions, methods and classes with their start and end lines, signature and leading comments.
5. **Chunk.** One chunk per symbol. A symbol that is too long is split into overlapping windows of about 60 lines. A file with no symbols is split the same way.
6. **Embedding text.** Prefix each chunk with `File: <path>` and `Symbol: <name>`, so the vector carries location and name as well as the code.
7. **Embed in batches.** Send batches to the embeddings API with backoff on rate limits. Skip any chunk whose `content_hash` already exists, so unchanged code is never embedded twice.

8. **Store.** Upsert into `code_chunks` with the vector, the `tsvector` for keyword search, and the commit SHA.
9. **Call graph.** While parsing, record imports and call expressions into `symbol_edges`. Resolving by name within the repo is approximate and good enough.

Incremental indexing

Store `last_indexed_sha` per repo and `file_hash` per file. On a push, run `git diff --name-status <last_indexed_sha> <new_sha>`. Re-embed added and modified files, delete chunks for removed files, and treat a rename as delete plus add. Update `last_indexed_sha` only after the whole batch succeeds.

Retrieval at review time

For every changed symbol, build a context pack from these sources:

Source	What it returns	Why
Changed code	Full source of the changed function from the head commit, plus the file's imports	The agent must see the whole function, not only the hunk
Call graph	Up to 5 callers and 5 callees	Shows who depends on this change and what it depends on
Semantic search	Top 8 chunks by cosine similarity to the changed code, excluding the same chunk	Finds similar patterns and existing helpers
Keyword search	Chunks that mention the changed identifiers (`tsvector`)	Catches exact names that embeddings can miss
Tests	Test chunks that reference the symbol	Shows expected behavior and what is already covered
Repo conventions	Snippets from README, CONTRIBUTING and lint config	Lets style comments match this repo's rules

Merge the semantic and keyword results with reciprocal rank fusion, remove overlapping chunks, and trim to a token budget (for example 6,000 to 8,000 tokens per agent call). Log which chunks were retrieved for each review: the dashboard shows them, and the eval suite measures how often the needed context was found.

10. Agents, tool calling and findings

Four specialist agents review the change in parallel inside a LangGraph graph, a verifier removes unsupported claims, and plain deterministic code, not the LLM, decides what gets posted.

The graph

1. **Plan.** Classify changed files by language and risk and decide which agents to run (skip the test agent for docs-only PRs).
2. **Run in parallel.** Bug agent, security agent, quality agent, test agent. Each gets the diff, its context pack and the static-analysis results, and has its own prompt and tool list.
3. **Aggregate.** Merge all findings into one list.
4. **Verify.** A skeptic pass re-reads each high-severity finding against the code and drops it if the evidence does not support it.
5. **Summarize.** Write the PR summary: what changed, the main risks, counts by severity.

Agent	Looks for	Typical tools
Bug	Logic errors, null and undefined handling, off-by-one, missing `await`, race conditions, wrong error handling	`read_file`, `find_callers`, `get_symbol`
Security	Hardcoded secrets, injection, unsafe deserialization, missing auth checks, risky dependencies	`search_codebase`, `run_linter` (Semgrep results)
Quality	Readability, naming, duplication, consistency with the repo's own style	`search_codebase`, `list_tests_for`
Test	Changed functions with no tests; proposes unit tests in the repo's framework	`list_tests_for`, `read_file`, `get_symbol`

Tools the agents can call

All tools are read-only, scoped to one repo and one commit, and have Zod-typed inputs.

- `search_codebase(query, k)`: semantic and keyword search over the index.
- `read_file(path, startLine, endLine)`: file content at the head commit.

- `get_symbol(name)`: definition of a function or class.
- `find_callers(symbol)` and `find_callees(symbol)`: from the call graph.
- `list_tests_for(path)`: test files that reference this file.
- `run_linter(path)`: normalized ESLint, Semgrep or Bandit results.
- `get_pr_context()`: title, description and commit messages.

There is deliberately no `post_comment` tool. Agents only return findings; your code posts them. That keeps a prompt-injection attempt from causing an action on GitHub. Limit each agent to a fixed number of tool calls (for example 8) and a token budget.

Finding format

Every agent must return JSON that matches the same Zod schema:

```
{
  "file": "src/auth/login.ts",
  "lineStart": 42,
  "lineEnd": 45,
  "severity": "high",
  "category": "security",
  "title": "Password compared with ==",
  "message": "Why this is a problem and when it triggers.",
  "evidence": "The exact code lines that show it.",
  "suggestion": "Optional fixed code.",
  "confidence": 0.82
}
```

If validation fails, retry once with the validation error in the prompt, then drop the finding. Never post unvalidated output.

Static analysis plus LLM

Run the static tools first and give their results to the agents as evidence. Static tools are precise but cryptic, and LLMs are fluent but sometimes wrong. A finding that both a tool and an agent report gets a confidence boost. Run analyzers with your own config files, never the repo's: an ESLint config is JavaScript and would execute repo code.

Prompt-injection defense

Treat everything from the PR (code, comments, descriptions) as untrusted data. The system prompt says to never follow instructions found inside code. Tools are read-only, secrets are never placed in context, and output is accepted only through the schema.

From findings to comments

1. **Fingerprint** each finding as a hash of file, category, normalized title and nearest symbol. It stays the same across pushes, so the same issue is never posted twice.
2. **Dedupe.** Merge findings from different agents on overlapping lines and the same category; keep the highest confidence and combine the evidence.
3. **Filter.** Drop confidence below the threshold (start at 0.6 and tune it with the evals), findings not on changed lines, style nits unless the repo opted in, and anything matching a fingerprint or category that users marked unhelpful.
4. **Rank** by severity weight times confidence. Post the top N inline (default 10); list the rest in the summary. Critical findings are always shown.
5. **Format** each comment with a severity label, title, explanation, evidence and, for safe one-line fixes, a GitHub `suggestion` block.

Stretch goal. Run the generated tests in a sandbox container and post a test only if it compiles and passes. That makes the test agent far more trustworthy.

11. AI evaluation

Build a benchmark of PRs with known bugs and report precision, recall and false-positive rate for every configuration of your reviewer, because these numbers are what turn this from a demo into an engineering project.

Simple ga: Mee AI entha correct ga review chestundo marks vesi chudadam. Bugs unna PRs ni istharu, AI entha pattukundi, entha thappuga cheppindi count chestharu.

Dataset

Start with 30 cases and grow toward 100. Mix three kinds:

1. **Real bug fixes.** Take bug-fix commits from open-source repos (BugsInPy, Defects4J and CVE fix commits are good sources). The code before the fix is the PR under review; the fix shows where the bug was.

2. **Injected bugs.** Take clean PRs and insert a known bug: off-by-one, missing `await`, null dereference, SQL injection, hardcoded secret, race condition.
3. **Clean PRs.** Changes with no bug, used to measure false positives.

Each case is a JSON file in `evals/dataset/` with the repo snapshot reference, the diff, and the expected findings (file, line range, category). Version the dataset and record the version in every run.

Metrics

Metric	Definition
Precision	Real findings divided by findings posted
Recall	Expected bugs found divided by expected bugs
False-positive rate	Average findings posted per clean PR
Location accuracy	Share of correct findings within 3 lines of the true location
Noise	Comments posted per PR
Cost and latency	Tokens, dollars and seconds per review
Retrieval hit rate	Share of cases where the context needed to spot the bug was retrieved

A finding matches an expected one when the file is the same, the line ranges overlap within the tolerance, and the category agrees. Use an LLM judge only as a fallback for semantic matches, and hand-label 20 cases to report how often the judge agrees with you.

Ablation table (the resume gold)

Run the same dataset through each configuration and publish the results in the README:

Configuration	Precision	Recall	FP rate	Cost per review
Diff only, one LLM call	measure	measure	measure	measure
+ RAG context	measure	measure	measure	measure

Configuration	Precision	Recall	FP rate	Cost per review
+ static analysis	measure	measure	measure	measure
+ multi-agent and verifier	measure	measure	measure	measure

Fill every `measure` cell with your real result. If one step does not help, write that honestly; it is still a useful finding.

Running evals

- `pnpm eval` runs the real pipeline with the GitHub layer stubbed (diff and repo snapshot are fed in), writes a JSON and markdown report, and stores a row in `eval_runs`.
- Fix the model version and use temperature 0 for repeatability. LLMs are still not fully deterministic, so run each case 3 times and report the average.
- In CI, `eval.yml` runs a 10-case subset on pull requests that touch prompts, agents or retrieval, and the full set nightly. It fails if precision drops more than a set margin below the stored baseline from `main`.

Online metrics

Once real users exist, track acceptance rate (helpful divided by helpful plus not helpful), the share of review threads marked resolved, and the share of comments that get no reaction. Show all of these on the dashboard.

12. Dashboard and feedback loop

The dashboard shows what the reviewer did and why, and its thumbs-up and thumbs-down buttons feed the loop that makes reviews quieter and more accurate over time.

Pages

Page	Shows	Build priority
Login	GitHub OAuth sign-in	1
Repos	Installed repos, index status and progress, last indexed commit, re-index button	1

Page	Shows	Build priority
Reviews	PR, status, time, finding count, cost; filters by repo and status	1
Review detail	Each finding with severity, file and line, evidence, suggestion, link to GitHub, thumbs up and down. A debug panel lists the retrieved context chunks	1
Settings	Per repo: confidence threshold, enabled agents, ignore globs, max comments, review drafts on or off	2
Insights	Findings by category over time, acceptance rate, average review time, cost per review, noisiest categories	3
Evals	Eval runs over time with precision, recall and false-positive rate	3

REST API

- `GET /api/repos`, `POST /api/repos/:id/reindex`
- `GET /api/repos/:id/settings`, `PUT /api/repos/:id/settings`
- `GET /api/reviews?repoId=&status=`, `GET /api/reviews/:id`
- `POST /api/findings/:id/feedback`
- `GET /api/insights?repoId=`, `GET /api/evals`

Every endpoint checks that the signed-in user can access the repo through an installation. Poll review status every few seconds or use server-sent events.

How feedback is collected

1. Thumbs buttons in the dashboard write to `feedback`.
2. A scheduled job polls the reactions on posted comments and stores them as feedback.
3. The `pull_request_review_thread` webhook reports resolved and unresolved threads. A resolved thread is a weak positive signal.

How feedback changes behavior

1. **Suppress repeats.** A finding whose fingerprint was marked not helpful is not posted again in that repo.
2. **Adjust thresholds.** If a category has under 30% acceptance over its last 20 comments, raise that category's confidence threshold for that repo.
3. **Teach the prompt.** Keep a few not-helpful examples per repo and include the most similar ones in the prompt as things not to flag.

Build the Reviews and Review detail pages first. They are enough for the demo and for debugging.

13. Reliability, security, testing and observability

This section is what separates a backend project from an LLM wrapper: every external call can fail, every tenant's code is private, and every claim about quality needs a test.

Reliability

- **Timeouts everywhere.** Every call to GitHub, the LLM and the embeddings API uses an `AbortController` with a time limit.
- **Rate limits.** Read GitHub's `x-ratelimit-remaining` and `retry-after` headers and back off, including on the secondary rate limit. Use a token-bucket or backoff for LLM and embeddings calls.
- **Circuit breaker** on the LLM provider: after repeated failures, pause that queue briefly instead of burning retries.
- **Health endpoints.** `/healthz` for liveness and `/readyz` that checks Postgres and Redis.
- **Cost guards.** A token limit per review, a file limit per PR (post a short 'PR too large' note above it), and a daily budget per installation.
- **Graceful shutdown** and restart-safe jobs: every stage can be repeated without duplicate comments because of fingerprints and unique constraints.

Security

Risk	Control
Forged webhooks	HMAC verification with constant-time comparison

Risk	Control
Leaked secrets	Secret manager or env vars, Zod-checked at boot, never logged, rotate the key if exposed
Over-broad access	Minimum app permissions; installation tokens scoped to one repo; tokens held only in memory or Redis with a TTL
Cross-tenant leaks	Every query scoped by `repo_id`; row-level security in Postgres; vector search always filtered by repo; one temporary directory per job
Code privacy	Tell users their code is sent to the LLM provider; per-repo opt-out; do not log full source; delete all data on `installation.deleted`
Prompt injection	Untrusted-data prompt rules, read-only tools, no action tools, schema-only output
Running untrusted code	Never execute repo code; run analyzers in a container with no network, resource limits and a timeout, using your own configs
Web attacks on the dashboard	httpOnly SameSite cookies, CSRF protection, helmet, rate limiting, strict CORS
Vulnerable dependencies	Dependabot, `npm audit` in CI, Trivy scan on images

Observability

- **Logs.** pino structured logs carrying `deliveryId`, `reviewId`, `repoId` and `stage`, so one review can be traced end to end.
- **Metrics.** `review_duration_seconds`, `queue_depth`, `job_failures_total`, `llm_tokens_total`, `findings_posted_total`, `github_rate_limit_remaining`.
- **Errors.** Sentry for unhandled exceptions.
- **Alerts.** Any job in the dead-letter queue, queue depth above a threshold, or review p95 time above target.

Testing

Level	What to test
Unit	Signature verification, Zod schemas, chunking (golden files), diff-to-line mapping, fingerprinting, ranking, token cache expiry
Integration	Testcontainers for Postgres and Redis, GitHub mocked with nock or msw, LLM stubbed with fixture responses. Send a recorded webhook and assert the exact comment request that would be posted
Contract	Save real GitHub payloads as fixtures and test your Zod schemas against them
End to end	A sandbox repo with the app installed; a script opens a PR and waits for the review. Run it manually or nightly
Evaluation	The eval suite from the previous section

Aim for at least 80% coverage on `core`, `github`, `retrieval` and the post-processing code, and say in the README that this is the target.

14. DevOps: Docker, CI/CD and deployment

One `docker compose up` should start the whole system locally, and every merge to `main` should test, build and deploy it without manual steps.

Docker

- **Images.** One multi-stage Dockerfile per app: `api`, `worker` and `dashboard`. The worker image also includes `git`, Semgrep, Python with Bandit and Ruff, and ESLint.
- **Hygiene.** Run as a non-root user, pin base image versions, keep a `.dockerignore`, and keep secrets out of image layers.
- **docker-compose (local).** Services: Postgres from the `pgvector/pgvector` image, Redis, `api`, `worker`, `dashboard`. Add a webhook tunnel (smee.io client or cloudflared) so GitHub can reach your laptop.
- **Config.** Ship a `.env.example` and validate all variables with Zod at startup.

CI pipeline (`ci.yml`, on every pull request)

1. Install with a pnpm cache.
2. Type check (`tsc --noEmit`) and lint.
3. Unit tests.
4. Integration tests with Postgres and Redis as service containers.
5. Build the Docker images and scan them with Trivy.
6. Upload coverage.

Eval pipeline (`eval.yml`)

Triggered when prompts, agents or retrieval code change, and nightly. Runs the eval suite, posts the metrics table as a PR comment, and fails if precision drops below the baseline by more than the allowed margin.

CD pipeline (`deploy.yml`, on merge to `main`)

1. Build images tagged with the git SHA and push them to GitHub Container Registry.
2. Run database migrations as a release step.
3. Deploy to staging and run a smoke test (send a signed test webhook and check for a 202).
4. After approval, deploy to production.

Keep two GitHub Apps and two environments: `staging` and `production`, each with its own secrets.

Hosting options

Option	Effort and cost	Good for
Render, Railway or Fly.io	Lowest effort; a few dollars a month	Getting a live demo up quickly
One AWS EC2 or VPS box running docker-compose	Medium effort; can fit free-tier credits	Showing real DevOps: Linux, Docker, reverse proxy, TLS
AWS ECS Fargate with RDS and ElastiCache	Highest effort and cost	A stretch goal after everything else works

Managed Postgres with pgvector (Neon or Supabase) and managed Redis (Upstash or Redis Cloud) keep the first deployment simple. Check current free-tier limits before you choose.

Extra credit

- **Dogfooding.** Install the app on its own repo so every PR to the project is reviewed by the project. Screenshot this for the README.
- **Backups.** Daily Postgres backups and Redis persistence if you want queued jobs to survive a restart.
- **Infrastructure as code.** Terraform for the AWS option.
- **Releases.** Semantic version tags, a changelog, and image tags by git SHA.

15. Build plan

Build it in four two-week phases, and expect to be resume-ready after week 2: a real PR that gets an AI comment already proves webhooks, auth, queues and an LLM working end to end.

Resume-ready after week 2, live product by week 8

Phase 1: Working review — Weeks 1 to 2

Docker compose, webhook server with HMAC check and idempotency, BullMQ queue
Database schema and migrations, Zod validation, structured logging
GitHub App auth (JWT to installation token), diff fetch, check run
One LLM review of the diff, posted back as an inline PR comment

Gate: a real PR gets an AI comment. Put the project on your resume.

Phase 2: Context and agents — Weeks 3 to 4

Tree-sitter chunking, embeddings, pgvector index, incremental re-indexing
Retrieval of callers, callees and similar code with a token budget
LangGraph agents (bug, security, quality, test) with read-only tools
Zod-validated findings with a repair retry for bad LLM output

Gate: retrieval finds the right callers on a test repo and findings validate.

Phase 3: Quality and evaluation — Weeks 5 to 6

ESLint, Semgrep and Bandit evidence; dedupe, ranking, confidence threshold
Verifier pass, comment cap, stale-review cancel, retries, dead-letter queue
Eval dataset of 30 or more PRs, metrics runner and the ablation table
Cost and latency tracking for every review

Gate: eval numbers exist for every configuration, including a diff-only baseline.

Phase 4: Product and delivery — Weeks 7 to 8

Dashboard: reviews, review detail with retrieved context, settings, feedback
Feedback loop, insights page and evals page
GitHub Actions CI and eval workflow, Docker images, staging and production deploy
README with diagram and results table, demo video, review your own repo with it

Gate: live URL, results table in the README and a 3-minute demo video.

build plan · 4 phases, 4 gates

Each gate is a check you can demonstrate, not a feeling. Do not start the next phase until the gate is passed.

If time is short. Phases 1 and 2 are the minimum for a strong project. Cut the dashboard's insights and eval pages first, then the feedback loop, and keep the eval suite: it is what makes the project stand out.

Working habits. Commit daily, write each ADR when you decide rather than later, update the README at every gate, and screenshot real PR comments as you go for the demo.

16. Resume, README, demo and interview prep

Put this project on your resume once a real PR gets a real AI review (the end of Phase 2), and fill every bracketed number with a result you measured yourself, because interviewers will ask how you measured it.

Resume entry

AI Code Review Platform | TypeScript, Node.js, PostgreSQL (pgvector), Redis/BullMQ, LangGraph, Docker, GitHub Actions | GitHub link | Live demo link

- Built a GitHub App that reviews pull requests automatically, with signed and idempotent webhook handling and a BullMQ job queue featuring retries, a dead-letter queue and cancellation of stale reviews when new commits arrive.
- Designed a RAG pipeline that parses repositories with Tree-sitter, embeds function-level chunks into pgvector with incremental re-indexing, and retrieves callers, callees and similar code for context-aware reviews across [N] files.
- Implemented a multi-agent LangGraph workflow (bug, security, quality and test agents) using tool calling, Semgrep and ESLint evidence, a verification pass and Zod-validated output, posting ranked inline comments through the GitHub API.
- Created an evaluation suite of [N] PRs with known bugs, reaching [X]% precision and [Y]% recall and reducing false positives by [Z]% against a diff-only baseline; the suite runs in CI to catch quality regressions.
- Containerized the system with Docker, automated test, build and deploy with GitHub Actions, and added structured logging and Prometheus metrics; deployed at [link] with a p95 review time of [T] seconds.

Keep it to 4 or 5 bullets. If a bullet describes something you have not built yet, leave it out.

README checklist

- [ ] One-line pitch and a demo GIF at the very top
- [ ] Architecture diagram (copy it from section 2)
- [ ] 'How it works' in six steps
- [ ] Quickstart: `docker compose up`, GitHub App setup, `.env.example`
- [ ] Eval results: the ablation table with the dataset size and date
- [ ] Screenshots of real inline comments on a PR and of the dashboard
- [ ] Design decisions linked to the ADRs

- [ ] Limitations and roadmap, stated honestly
- [ ] 2 to 3 minute demo video
- [ ] Live demo link

Three-minute demo script

1. Open a PR containing a seeded bug, such as a SQL injection or a missing `await`.
2. Show the webhook arriving in the logs and the job entering the queue.
3. Show the check run in progress, then the inline comments appearing.
4. Open the dashboard review page and show the retrieved context.
5. Push a new commit mid-review to show the stale review being cancelled.
6. End on the eval table.

Interview questions and short answers

Question	Short answer
Why a queue instead of working inside the webhook?	GitHub expects a reply in about 10 seconds and a review takes minutes. The queue adds retries, backoff, concurrency limits and survives crashes.
How do you prevent duplicate reviews?	Delivery id as a primary key, BullMQ job id of repo, PR and commit SHA, and a unique constraint on `reviews`. Fingerprints stop duplicate comments.
What happens when a new commit arrives mid-review?	The old review is flagged cancelled in Redis, LLM calls are aborted, and the head SHA is re-checked before posting.
How do you reduce false positives?	Static-analysis evidence, a verifier pass, a confidence threshold tuned on the evals, dedupe, a comment cap and feedback-based suppression.
How do you know the reviewer is any good?	An eval suite of known-bug and clean PRs reporting precision, recall and false-positive rate, with an ablation table, run in CI.

Question	Short answer
Why chunk by function?	A function is a unit of meaning. Line-based chunks cut functions in half and give worse embeddings and worse retrieval.
How do you isolate tenants?	Every query is scoped by repo, Postgres row-level security, one temp directory per job, per-installation tokens, deletion on uninstall.
What if the LLM returns invalid JSON?	Zod validation, one repair retry that includes the error, then the finding is dropped. Unvalidated output is never posted.
How do you handle prompt injection?	PR content is treated as untrusted, tools are read-only, there is no tool that posts or changes anything, and output is accepted only through the schema.
How do you control cost?	Token budgets, file and PR size limits, an embedding cache keyed by content hash, incremental indexing and a daily budget per installation.
How would it scale?	Workers are stateless and scale horizontally, queues can be partitioned per installation, and pgvector can be replaced by a dedicated vector database if the index outgrows it.
What would you build next?	Validated auto-fix suggestions, per-repo learning of conventions, more languages, and support for other Git hosts.